

Reviewer Pack — Minimal Diophantine Degree Correlation with Circuit Monotone...

Entry 50cf3cdf5d60 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 16:48:45 UTC

Minimal Diophantine Degree Correlation with Circuit Monotone Width

Entry ID: 50cf3cdf5d60 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 16:48:45 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Geometry) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

For every boolean circuit C , the minimal diophantine degree of a polynomial p representing C is linearly correlated with its monotone width $w(C)$, such that $d(p) = \Theta(w(C))$.

Rationale (proposer's reasoning):

The Diophantine geometry provides an algebraic way to encode computations, potentially revealing hidden structure in circuit complexity. A lower diophantine degree might correspond to a more compact or efficient circuit representation.

Taxonomy category: DIOPHANTINE_GEOMETRY_TO_CIRCUIT_COMPLEXITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 49d08e6292030850

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the monotone width and diophantine degree exceeds 0.7 for all 30 circuits, and no seed produces a correlation coefficient below 0.6.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Diophantine Degree AND Circuit Monotone Width - Diophantine Geometry IN BOOLEAN CIRCUIT COMPLEXITY - Correlation (Number Theory) WITH (Boolean Circuit Complexity)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_circuit(n):
        circuit = []
        for _ in range(2**n):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(n)]
            circuit.append((gate_type, inputs))
        return circuit

    def truth_table(circuit):
        n = len(circuit[0][1])
        table = {}
        for i in range(2**n):
            inputs = [(i >> j) & 1 for j in range(n)]
            output = evaluate_circuit(circuit, inputs)
            table[tuple(inputs)] = output
        return table

    def evaluate_circuit(circuit, inputs):
        stack = []
        for gate_type, inputs in circuit:
            if gate_type == 'AND':
                result = 1
                for input_val in inputs:
                    result &= input_val
                stack.append(result)
            elif gate_type == 'OR':
                result = 0
                for input_val in inputs:
                    result |= input_val
```

```

        stack.append(result)
    return stack[-1]

def diophantine_degree(truth_table):
    n = int(math.log2(len(truth_table)))
    matrix = []
    for i in range(2**n):
        row = [truth_table[tuple(inputs)] for inputs in itertools.product([0, 1], repeat=n)]
        matrix.append(row)

    # Gaussian elimination to find the rank
    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        rank = 0
        for i in range(cols):
            if all(matrix[j][i] == 0 for j in range(rank, rows)):
                continue
            matrix[rank], matrix[i] = matrix[i], matrix[rank]
            for j in range(rank + 1, rows):
                factor = matrix[j][i] / matrix[rank][i]
                for k in range(cols):
                    matrix[j][k] -= factor * matrix[rank][k]
            rank += 1
        return rank

    return gaussian_elimination(matrix)

def monotone_width(circuit):
    n = len(circuit[0][1])
    max_width = 0
    for i in range(2**n):
        inputs = [(i >> j) & 1 for j in range(n)]
        width = sum(1 for gate_type, _ in circuit if any(input_val == 1 for input_val in gate_
        max_width = max(max_width, width)
    return max_width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_random_circuit(n)
    truth_table_dict = truth_table(circuit)
    d_p = diophantine_degree(truth_table_dict)
    w_c = monotone_width(circuit)
    results.append((d_p, w_c))

if len(results) < 30:
    return {
        "metric_name": "correlation",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "insufficient_data"
    }
}

```

```

d_ps = [d_p for d_p, _ in results]
w_cs = [w_c for _, w_c in results]

mean_d_p = sum(d_ps) / len(d_ps)
mean_w_c = sum(w_cs) / len(w_cs)
correlation = (sum((d_p - mean_d_p) * (w_c - mean_w_c) for d_p, w_c in results) /
               math.sqrt(sum((d_p - mean_d_p)**2 for d_p in d_ps) *
                           sum((w_c - mean_w_c)**2 for w_c in w_cs)))

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": correlation >= 0.7 and all(corr >= 0.6 for corr in [correlation]),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"])
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results if result["metric_value"]))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and any(result["metric_value"] > 0 for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_below_threshold\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete its task and verify the conjecture. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17538
2	preregistration	ollama_remote	glm4:latest	0	0	16915
3	novelty	ollama_remote	glm4:latest	0	0	12461
4	novelty	ollama_remote	glm4:latest	0	0	12629
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23815
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23434
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15949
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21357
9	judge	ollama_remote	glm4:latest	0	0	15083

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 159180 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/50cf3cdf5d60.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/50cf3cdf5d60.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/50cf3cdf5d60.tar.gz (if generated)